pfi

Python for Fixed Income

COURSE 8 · IMPLEMENTING AGENTIC AI

Tools from Your Toolkit

WEEK 1 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: tools from your toolkit

Five bonds of the synthetic book, 2026-09-30

CUSIP	coupon	maturity	price	yield	OAS
99080CAE8	7.000	2032-06-18	106.727	5.606	147
99039NAC0	5.125	2030-06-03	98.730	5.509	153
99034IAA0	4.625	2038-09-17	88.692	5.960	155
99126WAE0	4.125	2029-07-16	98.979	4.516	62
99033HAF2	7.000	2050-02-26	107.310	6.393	183

Today

1. bond_tools: seven wrappers
2. JSON-friendly arguments
3. validate_arguments
4. Many tools at once
5. Descriptions as prompts
6. clip_result: short results

Every model response in the notebook is a saved response, replayed. A live run can differ.

The rules for agents

What an agent produces: text, every number from a tool

Allowed names

the agent returned

the saved run

the brief states

Never called

the agent's opinion or view

what the agent believes

the agent's advice

On the synthetic book: a description, never a recommendation

Allowed names

DV01 per 100 of face

position DV01 in dollars

the largest DTS contributions in the file

Never called

cheap, rich, or attractive

a trade idea or a recommendation

overweight or underweight

Your own functions as tools

bond_tools(): seven wrappers, each described in its unit

`price`	clean price, per 100 of face
`yield_to_maturity`	yield to maturity, in percent
`accrued_interest`	accrued interest, per 100 of face
`modified_duration`	modified duration, in years
`dv01`	DV01, price points per 100 of face
`spread_duration`	spread duration, in years
`dts`	DTS, years times bp

Each wrapper's three jobs

ISO date text in

the frozen bondmath function

a dict, the unit in its key

rounded to 6 decimals

The docstring is the prompt: tool_schema sends it as the tool's description.

JSON-friendly arguments

```
settlement  string  the settlement date as YYYY-MM-DD, for example 2026-10-02.
maturity    string  the maturity date as YYYY-MM-DD.
coupon_pct  number  the annual coupon in percent, 5.0 for a 5% coupon.
yield_pct   number  the yield to maturity in percent, 5.06 for 5.06%.
```

Three tool requests written by hand, through run_tools

Dates as ISO text	`dv01_per_100: 0.127083`
Slashes in a date	`error: dv01 failed: ValueError: '2026/09/30' is not a date written as YYYY-MM-DD`
Coupon as a decimal	`error: coupon_pct 0.07 is below 0.5: write the coupon in percent, 5.0 for a 5% co`

JSON has no date: dates cross as text, percents as numbers in percent. The wrapper converts.

Checking arguments before a tool runs

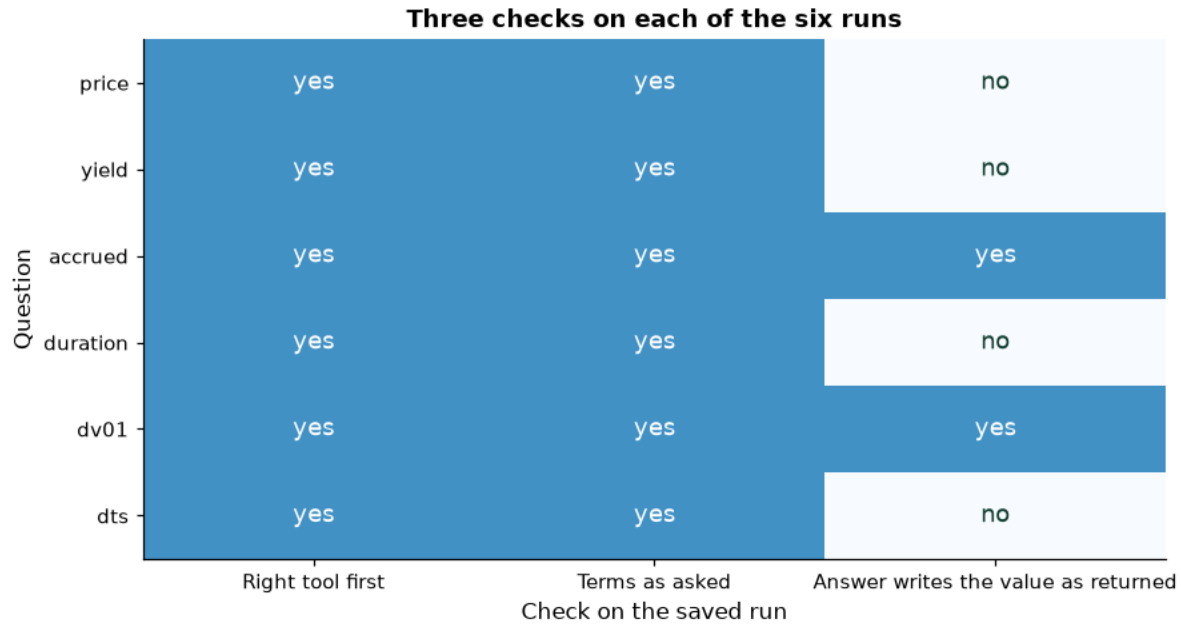
The same bad tool requests, to the plain dv01 and to the checked one

The yield as text	`plain:  unsupported operand type(s) for /: 'str' and 'int'` `checked: dv01_per_100: 0.127083`
The yield as a word	`plain:  unsupported operand type(s) for /: 'str' and 'int'` `checked: argument yield_pct: Input should be a valid number,`
The maturity left out	`plain:  dv01() missing 1 required positional argument: 'matu` `checked: argument maturity: Missing required argument`

checked(tool) sends the same schema for all 7 tools: yes

An error result that names the field is one a model can correct from.

Many tools at once



All seven tools offered

Right tool first: yes on all six

Terms as asked: yes on all six

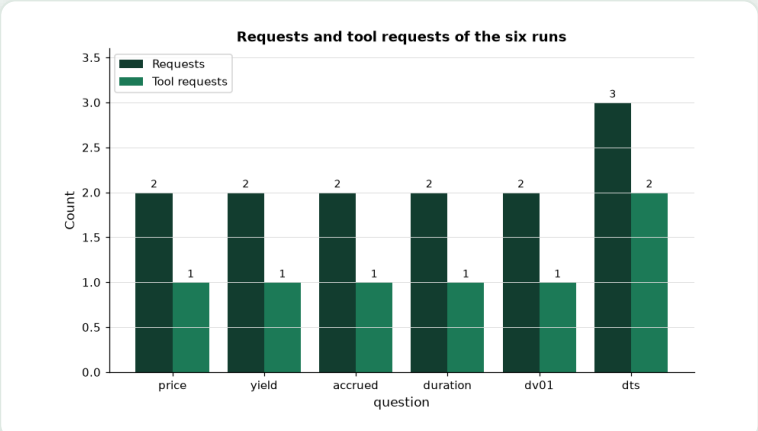
Written as returned: no on 4

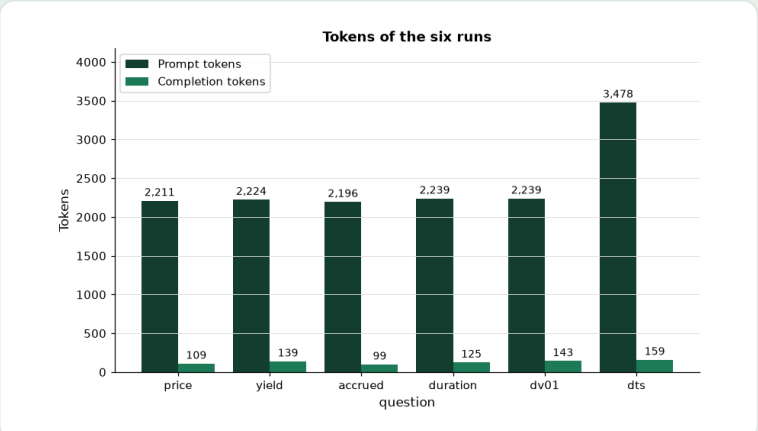
The answers rounded the tool's number

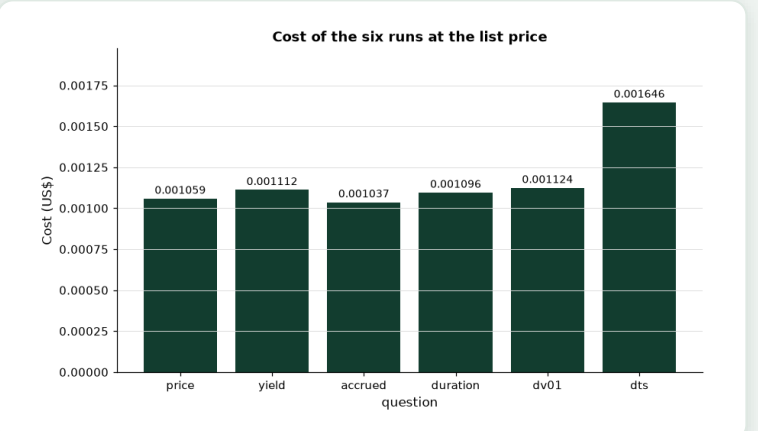
Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 2 requests, 2,211 prompt and 109 completion tokens; saved run recorded 2026-10-09; replayed (no key loaded).

The right tool on the right terms every time. The answers rounded 4 of the 6 numbers.

The six runs side by side







Question	Requests	Tool requests	Prompt tokens	Completion tokens	Cost (US$)
price	2	1	2,211	109	0.001059
yield	2	1	2,224	139	0.001112
accrued	2	1	2,196	99	0.001037
duration	2	1	2,239	125	0.001096
dv01	2	1	2,239	143	0.001124
dts	3	2	3,478	159	0.001646

Tool descriptions as prompts

One line of dv01's docstring, rewritten

```
- yield_pct: the yield to maturity in percent, 5.06 for 5.06%.

+ yield_pct: the yield to maturity as a decimal, 0.0506 for 5.06%.
```

The DV01 question, sent with each description

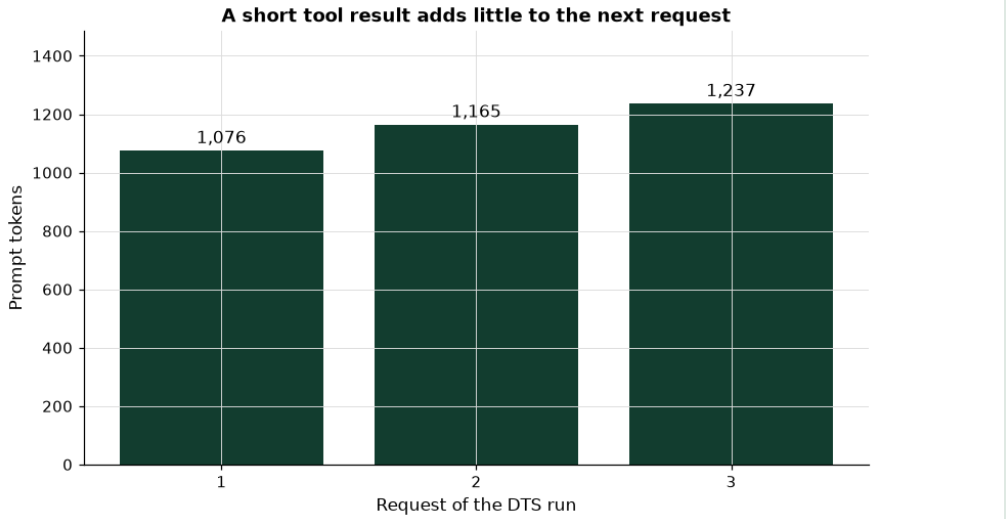
```
as written yield_pct sent 6.393; the tool returned 0.127083

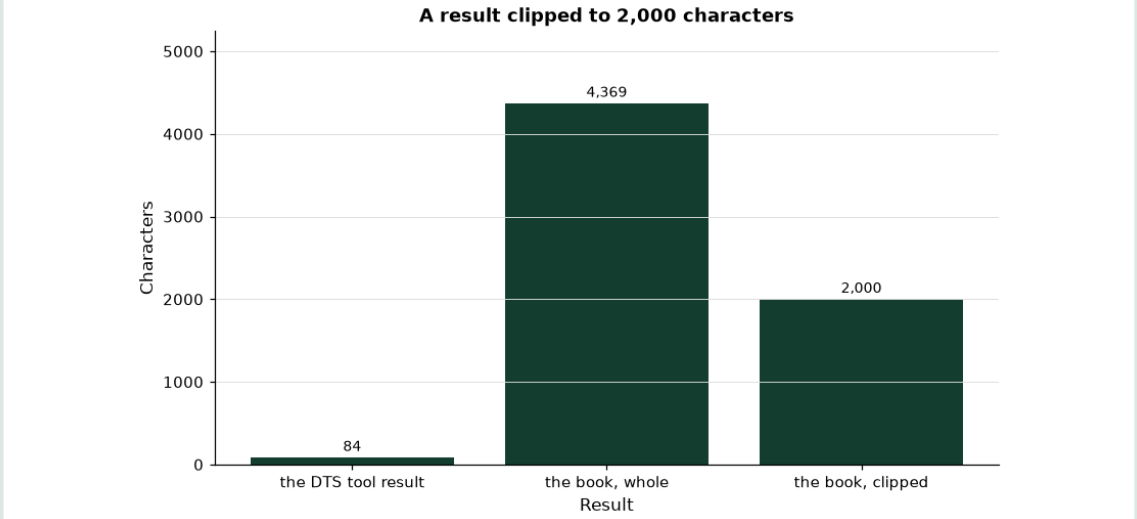
rewritten yield_pct sent 6.393; the tool returned 0.127083
```

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 2 requests, 2,245 prompt and 126 completion tokens; saved run recorded 2026-10-09; replayed (no key loaded).

The run did not change. A wrong description can pass every run: a test on the description catches it.

Keeping results short






```
clip_result(every_bond)  ends  ...906 [cut: 4,369 characters in all]
```

clip_result(result, max_chars=2000): compact JSON, floats at 6 decimals, a stated marker past 2,000

Every later request resends every result. A cut result is still partial: ask a tool for what you need.

A name for it

The arXiv record of 2302.04761

Title	Toolformer: Language Models Can Teach Themselves to Use Tools
Authors	eight, the first Timo Schick
Posted	9 February 2023; one version
Category	cs.CL, computation and language

Today, by hand: the model returns a tool request; your code runs your function on your machine.

Source, read 9 October 2026: export.arxiv.org/api/query?id_list=2302.04761 (the record, not the paper)

Excel and Git

Excel side by side (tested in Excel)

`=PRICE(...,0.07,0.05606,100,2,0)  (price)`	106.726595
`=YIELD(...,0.05125,98.73,100,2,0)*100  (yield)`	5.509049
`=100*0.04625/2*COUPDAYBS(...)/COUPDAYS(...)  (accrued)`	0.167014
`=MDURATION(...,0.04125,0.04516,2,0)  (duration)`	2.587515
`=MDURATION(...)*(PRICE(...)+accrued)/10000  (DV01)`	0.127083

the rewritten line: test_bond_tools_docstrings_state_units fails; git restore takes it back; 18 passed

Every Excel value equal to the saved run's tool result at 6 decimals (5 of 5). runs.csv: 7 runs.

The AI check

The docstring is the prompt. The agent code that comes back runs and looks right.

1

Docstring

A failure never stops the run, and a failure is never a number.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

A settlement date that does not exist. Run it: it fails.

→

4

Fix

Return an error result with the reason, as run_tools does.

Today: the except branch returns {"value": 0.0}: a failure that reads as a DV01 of zero.

Code that runs is not the same as code that is right.

What you can do now

- Wrap your functions as tools, units in every docstring
- Dates as text, percents as numbers
- Check arguments first: validate_arguments
- Read which tool the model requested, and with what
- Hold a description to its units with a test

Week 1, Day 4 Structured answers and traced numbers